University of Roehampton London

ASSESSMENT BRIEF

Programme	BSc (Hons) Computer Science		
Module Title	Full-Stack Web Development and Software Architecture		
Module Code	CMP020N302A		
Contact for queries about this assessment	*Dr. Marcus Vance (m.vance@roehampton.ac.uk)*		
Assessment Type(s)	Software Application, Technical Report, and Code Repository	**Word Length / Duration**	Codebase + 1,500-word Technical Documentation
Deadline (date & time) for Submission	Friday, 11 December 2026	**% contribution to module mark**	60%
Assessment Feedback date:	15 January 2027	**Format/Location of submission**	GitHub Classroom Repository Link & PDF Report via Turnitin

Assessment Task Requirements

Details of the Assessment Task

You are required to design, implement, and document a secure, full-stack RESTful web application using a modern web framework stack (e.g., React/Node.js or Next.js with PostgreSQL/MongoDB). The application must include user authentication, role-based access control, CRUD operations, API validation, and automated unit testing. Additionally, you must produce a technical documentation report detailing system architecture, API endpoints, database schemas, and deployment configurations.

Assessment Medium

Fully functional Git repository containing source code and automated test suites, accompanied by a structured PDF technical architecture report.

Essential Elements

- Initialize and push source code to the official **GitHub Classroom repository** provided for this module with atomic commits.
- Build a responsive front-end interface integrated with a backend RESTful API.

- Implement token-based authentication (JWT or OAuth2) and proper password hashing (bcrypt/Argon2).
- Write automated unit and integration tests achieving at least 70% code coverage.
- Include clear system setup instructions via a README.md and standard Dockerfile or configuration scripts.

Submission Links & Guidance

- Access your individual code assignment via the GitHub Classroom Module Portal.
- Submit your technical documentation PDF to the **Turnitin Submission Link** on the CMP020N302A Moodle page.

Assessment Success Guidance

Key Tips for Success

1. Architectural Separation: Ensure clean separation of concerns between your front-end presentation layer, backend routes, controller logic, and data storage layers.
2. Security First: Sanitize all input fields to prevent SQL injection and Cross-Site Scripting (XSS); never commit sensitive secrets or API keys to repository history.
3. Comprehensive Testing: Provide test suites for major API endpoints, covering both successful queries and boundary/error cases (e.g., 400 Bad Request, 401 Unauthorized).
4. Git Hygiene: Demonstrate consistent version control practices through meaningful commit messages and functional feature branches rather than a single bulk upload.
5. Clear Documentation: Structure your technical report with system diagrams (UML/ERD) to visually map out data models and component architecture.

Resources & Exemplars

- Starter templates and architecture diagrams are available on the Module Moodle Resources Section.

Layout and Style guidance

Layout Requirements

Your technical report must adhere to the following layout:

- System Architecture & Data Modelling (Include Entity-Relationship Diagrams & UML diagrams)
- REST API Endpoint Specifications (HTTP methods, request payloads, response codes)
- Security & Authentication Strategy (Data encryption, CORS, route protection mechanisms)
- Testing & Quality Assurance (Coverage summary, test run results, and validation)
- Deployment & Setup Guide (Environment variables, installation steps, build instructions)

Formatting Expectations

- Font: Consolas / Courier New for code listings; Calibri or Arial 11pt for body text
- Spacing: 1.5-line spacing
- Margins: Standard 1 inch (2.54 cm)
- File Format: Source code in GitHub; documentation as PDF or Word (.docx)

Referencing

Reference Style Requirements

This module requires IEEE Referencing Style for code libraries, framework documentation, and technical literature.

- In-text example: "As noted by M. Fowler, microservices allow decoupled deployment cycles [1]."
- Reference list example: [1] M. Fowler, Building Microservices, 2nd ed. Sebastopol, CA: O'Reilly Media, 2021.

Refer to the Roehampton Library IEEE Style Guide for formatting rules.

AI Referencing

The use of AI tools (e.g., GitHub Copilot, ChatGPT) for code generation or debugging is permitted but must be declared. Include an AI Usage Appendix in your report providing key prompts used, lines of code impacted, and evidence of manual code review and validation.

Learning Outcomes

By successfully completing this assessment, you will be able to:

- **LO1:** Design and develop full-stack software applications utilizing industry-standard design patterns and modern web frameworks.
- **LO2:** Implement robust cybersecurity principles, including identity management, input validation, and secure data storage.
- **LO3:** Apply software engineering practices including version control, automated testing, continuous integration, and clear technical documentation.

Employability and Professional Skills

By successfully completing this assessment, you will demonstrate:

- **Software Engineering Practice:** Mastery of industry-standard tools (Git, REST APIs, databases, automated test runners) used in modern dev environments.
- **Problem Solving & Security Awareness:** Ability to audit system vulnerabilities and write resilient code against common web exploits.
- **Technical Writing:** Skills to communicate complex technical designs, schemas, and system workflows to fellow developers and project stakeholders.

Marking Criteria/Assessment Rubric

Assessment name		**Full-Stack Software Project & Report**							
Assessment Criteria and LOs	Weighting if appliable	100 Exemplary	80+ Outstanding	70–79 Excellent	60–69 Very Good	50–59 Good	40–49 UG Adequate	30–39 Limited Fail	0–29 Limited/No Evidence Fail
Criterion 1: Application Architecture & Implementation (LO1)	30%	Exceptionally engineered, production-ready system exhibiting exemplary design patterns and performance.	High-quality, robust codebase with clean architecture and fully functioning features.	Solid application implementation; minor edge cases or UI/UX flaws present.	Functional system with minor bugs; acceptable adherence to framework patterns.	Basic working application; incomplete features or messy architecture.	Barely functional system; major bugs and poor design patterns.	Incomplete application with severe structural flaws.	Non-functional or unsubmitted application code.
Criterion 2: Security & Data Management (LO2)	20%	Flawless security controls, comprehensive data validation, and optimal database indexing and schema design.	Strong implementation of authentication, authorization, and input sanitization.	Effective security and data handling with minor configuration oversights.	Basic security implemented (e.g. standard auth); minor validation gaps.	Functional database connection but minimal security or missing route protection.	Vulnerable implementation; weak auth or lack of input validation.	Major security flaws (e.g. plain text passwords, open SQLi).	Total lack of data validation or security implementation.
Criterion 3: Testing & Version Control (LO3)	20%	Flawless Git workflow with clear commit history; comprehensive test suite	Excellent Git commit frequency and detailed unit/integra	Good test coverage (>70%) and logical Git history and branching.	Adequate unit tests present; git history shows bulk commits	Minimal tests written; basic git usage with sparse	Few or non-functional tests; code submitted in a single bulk commit.	Missing automated tests; chaotic source control.	No tests or version control evidence provided.

		exceeding 85% coverage.	tion testing suites.		near deadline.	commit logs.			
Criterion 4: Technical Documentation & Diagrams (LO1, LO3)	20%	Professional-grade technical report with clear ERD/UML diagrams and exhaustive API specs.	Clear, highly detailed documentation with accurate system diagrams and setup steps.	Well-structured report detailing endpoints and schemas with minor omissions.	Complete documentation but lacking visual clarity in architecture diagrams.	Satisfactory report; missing detailed API specifications or setup instructions.	Minimal report provided; brief setup notes and weak architectural diagrams.	Very poor documentation; lacks core sections or clear system explanations.	Unsubmitted or completely inadequate documentation.
Criterion 5: Code Quality & Formatting (LO3)	10%	Fully linted, clean code following strict conventions with comprehensive online documentation.	Highly readable code following consistent formatting and clear variable naming.	Well-organized code with minor formatting inconsistencies.	Acceptable code readability; sparse inline documentation.	Disorganized code structure; inconsistent indentation and style.	Hard-to-read code; lacks comments or basic naming conventions.	Extremely messy codebase; unformatted and difficult to parse.	

Use of Artificial Intelligence (AI)

Please see University guidelines here:

https://roehamptonprod.sharepoint.com/sites/portal/information/LTEU/Documents/4a%20AI%20principles%20and%20guidance%20-for%20Senate.docx

- The assessment is designed so that the use of AI during the assessment is possible. You must acknowledge any use of AI and appropriately cite all AI generated outputs. Please make sure you read and understand the assessment guidelines and ask your Module Leader if you have any questions. You can find the guidance on the use of AI here*.

Ethical Requirements

All software developed within this assessment must adhere to data privacy regulations (e.g., UK GDPR). Applications must use synthetic, dummy data during testing and development. Real-world personal user data must not be stored or processed within public GitHub repositories or test environments.

Submission Checklist

Before you submit, check:

- Have I fully answered the assessment brief?
- Have I met the word count and formatting requirements for the technical report?
- Is my referencing complete and accurate according to IEEE guidelines?
- Have I declared any AI use honestly in the appendix?
- Have I proofread my work and tested my code build?
- Am I submitting through the correct platforms (GitHub Classroom & Turnitin) before the deadline?

Additional Information

Mitigating circumstances/late penalties

Sometimes circumstances outside of your control may affect your studies and might prevent you from submitting work on time or attending an exam. To avoid late submission penalties, look at our options on the mitigating circumstances portal.

Please speak to your Personal Tutor, the Roehampton Student Union (RSU) or someone in the Wellbeing team first, who can support you.

Resubmissions and Reassessment

If you are required to resubmit this assessment or take part in reassessment, you will be notified via Moodle and your student email. Please ensure you check both regularly. Any reassessment tasks will follow the same learning outcomes and criteria.